Fachhochschule Salzburg GmbH
Fachbereich, Software

Project Requirements September 3, 2025

Software Design - Lab

Project: **Ticketing (Soccer)**

Henrik Binggl Bernhard Danninger Josef Lettner

Lab Project:

In this lab you will work in teams of two to a maximum of three students and develop a small software project that will be planned, implemented, tested, and presented over the course of the semester. Each team will implement two use cases (three use cases in the case of a three-person team). Ideally, a team should consist of one designer and one developer, in order to combine different perspectives within the project work. In the project description, requirements marked with **Team (3)** are intended specifically for teams of three students.

The lab follows the typical development process of a software system. Starting with the analysis of requirements, you will create models such as use case, class, component, activity, and sequence diagrams. These models will form the blueprint for your implementation. Each team will be provided with a starter project in the chosen technology stack, which you will gradually extend by adding the designed components, persistence layer, and services.

Validation of your implementation through systematic testing is an essential part of the lab. You will create unit tests and apply mocking techniques where appropriate, ensuring that the logic of your system corresponds to the requirements specified in your models. In addition, you will apply relevant design patterns and reflect on how non-functional requirements can act as architecture drivers in your project.

Throughout the lab you will take on different roles: as analysts and designers you will create and refine models, and as developers and testers you will implement and validate them. While the implementation may sometimes follow with a certain time lag, each design activity is accompanied by coding tasks, helping you translate architectural decisions into working software and evaluate their impact in practice.

The overall aim is to deepen your understanding of the software development process by experiencing the interplay between modelling, implementation, testing, and architectural design.

Contents

1 Procedure **3**

2 Overview **3**

2.1 Ticket Types 4

2.2 Customers 5

3 Data Model **5**

3.1 Master Data 5

3.2 Transactional Data 6

4 Use Cases **7**

4.1 Single-Match Ticket Sale 7

4.2 Access Control 7

4.3 Season Ticket Sale 8

5 Expected Results and Quality Assurance **8**

6 Submission **9**

Team(3)

1 Procedure

When working on the individual models, it will certainly help if you are somewhat familiar with the application domain or take the time to read up on it. Most of you probably already have some experience with similar contexts — for example, having a bank account, borrowing books from a library, using a paid parking lot, or visiting a stadium. Pay attention to how certain processes are carried out in reality; this will help you approximate a workable solution. If needed, you may also do some research or ask around: for instance, what exactly happens when you purchase a ticket, borrow a book from the library, or buy shares on the stock market?

In the following sections you will find the project requirements as provided by the customer. Be aware that customers often use their own terminology, which may differ from the technical language we apply in software design. This is a typical situation in real-world projects, where part of your task as analysts and designers is to interpret and refine the given information.

As in many real projects, the initial requirements are not always fully complete and may leave room for interpretation. Try to close potential gaps in a way that seems most reasonable to you. If uncertainties remain, clarify them with the customer or stakeholders. In our lab setting, your "customer" may also be represented by the instructor, so do not hesitate to ask if questions arise.

2 Overview

We are a Premier League football club and own a 32,000-seat, all-seater stadium. This season, for the first time in our long history, we have qualified for the UEFA Champions League. We seek to design and implement an extensible, server-centric ticket shop system to support domestic league and European fixtures.

Background & Reason

Customer Features

- ☐ The system shall allow customers to purchase tickets, limited to a maximum of 4 tickets per person.
- ☐ The system shall allow season ticket holders to reserve tickets, limited to a maximum of 4 tickets per person.
- ☐ The system shall allow customers (or authorized staff) to cancel reserved tickets.
- ☐ The system shall support payment by cash (at staffed stalls only) and by credit card (online or at point of sale).

Client Interactions (POS, web, kiosk, app) Each client operates independently and communicates with the central server. A client shall:

- ☐ query the server for available seats within a specified sector/section,
- ☐ notify the server when a ticket/seat is sold,

Als erstes Prototyp Stadium mit 32.000, dann mehrere Stadien für alle Vereine der österreichischen Bundesliga (Kapazität anpassen an größtes Stadium Österreichs)

- ☐ notify the server when a ticket/seat is reserved,
- ☐ notify the server when a reserved ticket/seat is cancelled,
- ☐ request billing information for the current transaction.

Server Responsibilities The server shall provide:

- ☐ real-time status of seats (free, reserved, sold) per match and sector/section,
- ☐ billing and pricing information required to complete transactions,
- ☐ utilization statistics over a selectable period (e.g., single match, full season, winter season, spring season, Champions League fixtures).

2.1 Ticket Types

The stadium is divided into eight seating sections (A–H) and multiple price categories. Seat availability and pricing depend on the section and the match (league vs. European fixtures). The capacities per section are shown below.

Section	Capacity
A	8,000
B	8,000
C (Fan sector)	4,000
D	4,000
E	2,000
F	2,000
G	2,000
H	2,000

Price Categories. (Indicative levels: = lowest, = medium, = highest)

- ☐ **Category 1** — Sections A, B:
- ☐ **Category 2** — Section D:
- ☐ **Category 3** — Sections E, F, G, H:
- ☐ **Fan Sector** — Section C (fan sector):

Domänen übliche Preise

Regular Ticket A single-match ticket valid for one specific fixture (seat in a selected section and category).

Season Ticket A ticket valid for all home league fixtures in the specified season (assigned seat/sector per club policy). Season-ticket holders may also reserve regular tickets for individual matches subject to availability and club limits.

Champions League Ticket A ticket valid for UEFA Champions League home fixtures (match-specific or bundle, depending on competition phase).

Fan Ticket A discounted single-match ticket valid exclusively for the fan sector (Section C). Eligibility is restricted to members of the FC ITS Fan Club; membership verification is required at purchase and entry.

2.2 Customers

We distinguish the following customer types:

Regular Customers Pay the regular price in any stadium section.

Reduced-Price Customers Pay half price in all sections. Eligible groups are children (under 16 years) and persons with disabilities.

FC ITS Fan Club Members Are eligible to purchase tickets for the fan sector (Section C).

3 Data Model

In software systems, the distinction between *master data* and *transactional data* plays a central role. These two concepts form the foundation for structuring and managing information in many business and application domains.

3.1 Master Data

Master data refers to core and relatively stable information that provides the basis for a wide variety of processes within a system. Such data changes infrequently compared to other types of information.

Typical examples of master data in an enterprise context include information about products, customers, suppliers, or employees. This information is essential for daily operations and often serves as reference data for other processes.

Master data provides the backbone for many processes such as sales, purchasing, or payroll. It is therefore critical that this data is consistent, accurate, and up to date, since errors or inconsistencies can affect multiple parts of a system. For this reason, master data management is often given particular attention in the design of information systems.

Note that the boundary between master data and transactional data is not always perfectly clear. In this project description, some ambiguity is intentional in order to make the requirements realistic and closer to practice.

The following statements about master data are provided for your project:

- ☐ The system shall represent one or more stadiums operated by RB Salzburg; each stadium shall be identified by a name and a postal address.

- ☐ Each stadium shall be partitioned into sectors (sections), and each sector shall have its own seating capacity.
- ☐ Every match shall take place within a *competition/series* (e.g., Bundesliga, Cup, UEFA Europa League, UEFA Champions League, friendly).
- ☐ A competition/series shall consist of multiple matches and may recur each season (annually).
- ☐ A match shall be defined by its kickoff date and time, the stadium in which it is played, and the competition/series it belongs to.
- ☐ Exactly two teams shall participate in each match.
- ☐ The system shall explicitly model customer types (regular, reduced-price, and fan club member) and shall associate each customer with exactly one type. Team(3)

3.2 Transactional Data

Transactional data usually refers to information that records events, activities, or changes within a given context. Unlike master data, it is dynamic in nature and documents specific operations or interactions.

For example, transactional data could include customer orders, reservations, deliveries, or payments. This data changes frequently and captures the dynamic aspects of business processes.

The following statements about transactional data are provided for your project:

- ☐ The system shall sell tickets for RB Salzburg home matches; each ticket shall refer to exactly one *seat*.
- ☐ When a match is created, its initial status shall be `planned`. Spectator entry shall be permitted only when the match status is `active`. After the match is finished, the status shall be set to `conducted`; a match may also be set to `cancelled`.
- ☐ For any given match, the system shall recognize *single-match tickets* (valid for exactly one match) and *season tickets* (valid across the competition/series) as admission-entitled.
- ☐ Both single-match and season tickets shall be bound to a fixed, pre-defined seat.
- ☐ Ticket pricing (for both single-match and season tickets) shall be determined by *sector/section* and by *competition/series type*.
- ☐ The purchase of season tickets shall require customer registration (customer account).
- ☐ Single-match tickets shall be sellable anonymously at staffed ticket counters.
- ☐ For marketing purposes, the system shall record each ticket *usage* (stadium entry scan)

and shall provide functionality to associate ticket usages with customer identities where possible.

- ☐ For every conducted match, the system shall maintain statistics including: number of tickets sold, number of tickets used (entries), and occupancy (percentage of capacity).
- ☐ For every competition/series, the system shall maintain cumulative usage statistics; match statistics shall be added to the series statistics only after the match status is `conducted`.
- ☐ The system shall enforce ticket purchase rules based on customer type (e.g., reduced price only applied if the customer belongs to an eligible group, fan club members restricted to Section C).

Team(3)

4 Use Cases

Each project member is to implement one use case within the project. The selection of use cases is **not arbitrary**; they must be chosen in the order of the list below. Implementing alternative use cases is only possible **after** consultation with the instructors.

4.1 Single-Match Ticket Sale

The following business logic is to be implemented:

- ☐ The system shall allow a customer to select one of the upcoming matches for purchasing a ticket.
- ☐ The system shall allow the customer to choose a sector and a seat.
- ☐ The system shall display only *available* seats; for a seat to be available, there shall be
  - ☐ no single-match ticket already sold for the selected match, and
  - ☐ no season ticket already assigned for the competition/series covering that seat.
- ☐ The system shall display the ticket price at the appropriate step of the purchase flow.
- ☐ The system shall enforce concurrency control during ticket purchase to prevent double-selling the same seat.

4.2 Access Control

The following business logic is to be implemented:

Kunden loggen beim rausgehen, damit die gleiche ticket nummer nochmal rein kann.

- ☐ The system shall allow a customer to attempt entry by presenting a ticket number.
- ☐ The system shall grant entry only if:

Bei gleicher ticketnummer reingehen auch loggen, bei mehrfach gleicher nummer staff verständigen.

  - ☐ the ticket is valid for the *currently active* match (single-match ticket), or

- ☐ the ticket is valid for the *competition/series* of the currently active match (season ticket), and
- ☐ the ticket has not already been used for entry to this match.

4.3 Season Ticket Sale

Team(3)

The following business logic is to be implemented:

- ☐ The system shall allow a customer to select one of the available competitions/series for purchasing a season ticket.
- ☐ The system shall allow the customer to choose a sector and a seat.
- ☐ The system shall display only *available* seats; for a seat to be available, there shall be
  - ☐ no single-match ticket already sold for the relevant match, and
  - ☐ no season ticket already assigned for the selected competition/series.
- ☐ The system shall display the ticket price at the appropriate step of the purchase flow.
- ☐ The system shall enforce concurrency control during season ticket purchase to prevent assigning the same seat to multiple customers.

5 Expected Results and Quality Assurance

By the end of the semester, each team shall deliver a working system that supports the end-to-end execution of the assigned use cases under realistic, multi-user conditions and with all relevant constraints and rules enforced. A student-facing web application shall provide the core user journey for discovering availability and placing bookings (i.e., searching and reserving items and any applicable services). In domains that involve a physical device (e.g., an ATM or a beach chair), a lightweight HTML page shall be provided to simulate the device: it must drive the same server-side flows as the real device would (for example, entering a reservation or card number, requesting unlock/deny, showing current status, and triggering state transitions).

The implementation is expected to respect concurrency and data-integrity concerns (no double booking, correct handling of overlapping requests, capacity limits, pricing or tournament rules, etc.). A bespoke CRUD user interface for master-data maintenance is *not* required; seed data, fixtures, or migration scripts are sufficient to populate master data needed for demonstrations and tests. The system shall include automated unit tests for the critical paths (reservation creation and modification, concurrent requests, device unlock/deny, capacity and rule enforcement), and the project shall document how to run these tests. During the lab we will review sound unit-testing practices (isolation, Arrange–Act–Assert, mocks/stubs, and boundary/error cases), and your test suite is expected to reflect these practices alongside basic input validation and error handling.

At submission time, each team shall hand in: (1) the implementation (source code and minimal

setup/run instructions), (2) concise documentation covering the architecture, data model, and external API sufficient for another team to run and test the system, and (3) a short screencast of approximately five minutes *per use case* showing the application working in your environment and briefly explaining a code fragment that you consider important.

6 Submission

All submissions must be made exclusively via Moodle. Ensure that the names of all project members are included in every submission (e.g., on the cover page and in source code comments). **Create and maintain a single consolidated project report that you update across all deliverables. Always submit your consolidated report as a PDF file.**